



Production, Manufacturing, Transportation and Logistics

Fragment-based solution of two skip pickup and delivery problems

Michael Forbes^a, Thomas Whitaker^a, Sanne Wøhlk^{b,*}^a School of Mathematics and Physics, The University of Queensland, Australia^b Department of Business Development and Technology, Aarhus University, Denmark

ARTICLE INFO

Keywords:

Pickup and delivery
Integer programming
Branch and cut

ABSTRACT

We present a new fragment-based model for two versions of the Skip Pickup and Delivery Problem (SPDP). Our new algorithm outperforms the state-of-the-art for these problems by at least two orders of magnitude. In these particular SPDPs, skips must be picked up from their origin, emptied at various treatment facilities, and returned to their origin or to a different delivery location, subject to certain balancing constraints. The problem, which has applications in the waste and building industries, is to create route plans for vehicles of capacity two. We model the problem using an extension of the recently proposed method of fragments for Pickup and Delivery Problems (PDP) and integrate it in a Branch and Cut algorithm. Our algorithm is generic enough to solve both versions of the problem with the addition of a single constraint. For both versions of the problem, we identify and prove many new optimal solutions and significantly reduce runtime for the instances already solved.

1. Introduction

In this paper, we study two versions of a Skip Pickup and Delivery Problem (SPDP): one with fixed return of empty skips (SPDP-R) and one where the return of skips is flexible (SPDP-F). Both versions were first presented by Wøhlk and Laporte (2022).

The SPDP originates from a practical application where citizens sort their reusable and recyclable material into skips at recycling centers. The skips are picked up from recycling centers and transported to a content-dependent treatment facility, where they are emptied. Finally, the skips are returned to a recycling center to be available to citizens. Thus, each request consists of three actions: pickup, treatment, and delivery. In the SPDP-R, the delivery location of each skip is the same as its pickup location. In the flexible return case (SPDP-F), the return of the skips can be to different recycling centers provided that each recycling center ends the day with as many empty skips of each type (high, low, with lid, etc.) as they had picked up during the day. In the practical applications, skips are transported by vehicles of capacity two and the vehicles perform open routes with route duration limits and with a fixed cost of using each vehicle.

Three characteristics make the SPDP significantly different from traditional Pickup and Delivery problems. Firstly, in the SPDP, the requests consist of three actions rather than two. Secondly, in the SPDP, the number of locations is limited, causing several requests to be identical. For example, instance B20 has 63 requests, but covers only seven pickup and delivery locations and five treatment facilities. Consequently, the prob-

lem can be viewed as having 23 unique requests, each of which must be serviced multiple times (see Table 4). Contrary to previous studies, this is how we view the problem. Thirdly, the SPDP-F has the additional characteristic of flexibility, which is not common in traditional problems.

Only one other paper (Belenguer et al., 2024) has presented exact solutions of this problem. However, both papers only solve very small instances. In fact, the largest instances solved to optimality within two hours in Wøhlk and Laporte (2022) using an IP model have 12 skips for fixed return and 7 skips in the flexible return case. Belenguer et al. (2024) focus on the fixed return case. The largest instance they solve within one hour has 15 skips using a Branch and Cut approach. In this paper, we apply a completely different modeling technique based on fragments (Alyasiry et al., 2019) to develop significantly stronger exact algorithms for both versions of the problem. The largest instances we can solve within one hour has 40 skips for the fixed return case and 49 skips for the case with flexible return of skips. We thereby show how powerful fragments are when solving this difficult problem.

In the last few years a new method has emerged to exactly solve some classes of PDPs. Known as the method of fragments, it was proposed by Alyasiry et al. (2019) and first applied to the PDP with time windows and last-in-first-out loading (PDPTWL). Since then the method has been developed and applied to the dial-a-ride-problem (DARP) (Rist & Forbes, 2021, 2022). In all three cases, it proved to significantly outperform state of the art for those problems, which motivated us to apply the method to the SPDP-R and the SPDP-F. Due to the lack of time windows

* Corresponding author.

E-mail addresses: m.forbes@uq.edu.au (M. Forbes), sanne@btech.au.dk (S. Wøhlk).

and the complex state-spaces of the two SPDPs, we integrate the method of fragments into a Branch and Cut algorithm rather than, for instance, a Branch and Price algorithm.

The main contributions of this paper are: (1) We are the first to apply a fragment-based approach to a pickup and delivery variant with more than two events per request; (2) We show how a combination of a fragment-based model and redefining requests as sets of identical skips provides a model which is both simpler and stronger than state-of-the-art; (3) We develop a model which can be easily modified to solve both variants of the problem, and which could easily be extended to other potential variants; (4) To the best of our knowledge, this is the first time the method of fragments is applied to a problem, where requests are covered multiple times. Our choice of modeling the problem in this way, is a key reason for the success of our approach. We present special cuts to handle sub-tours in this case; (5) We solve the SPDP-R about a factor 35 faster than the state-of-the-art Branch-and-Cut, prove optimal solutions for 31 instances (23 for the first time) and additionally provide improved solutions for 10 instances (of 49 instances); (6) For the SPDP-F we prove optimal solutions for 32 instances (30 for the first time) and additionally provide improved solutions for 6 instances (of 49 instances).

The paper contains the following sections. Section 2 provides a brief overview of the most important literature. Section 3 gives formal definitions of both the SPDP-R and the SPDP-F. Section 4 first provides an intuition of the network underlying our model, and then provides the technical definition of it. Section 5 provides the model for SPDP-R while Section 6 explains the modifications needed to model the SPDP-F. Section 7 gives an overview of our algorithmic enhancements. Section 8 provides computational insight and compares our results to the previous attempts to solve the problem. Section 9 concludes the paper. Finally, Appendix A provides an overview of the notation used to model and solve the problems.

2. Literature review

In this section, we review the most relevant literature for our paper. We will focus on SPDP in Section 2.1 and on fragment-based methods in Section 2.2. The problem we study originates from real life waste operations. We refer to Hess et al. (2023) for a recent survey on waste collection routing challenges.

2.1. Skip pickup and delivery

The SPDP belongs to the large class of Pickup and Delivery problems (PDP). A special feature of the SPDP compared to other PDPs is that, due to practical limitations, the vehicles can only transport one or two skips simultaneously. For excellent classifications and surveys of PDP, we refer the reader to Berbeglia et al. (2007), Parragh et al. (2008), and Battarra et al. (2014). In the classification of PDPs, both problems studied in this paper are one-to-one problems. However, unlike traditional PDPs, our problems consist of triplets of events (pickup, treatment, and delivery). The SPDP-R is double-paired and the SPDP-F is first paired (pickup to treatment) and then paired with flexibility (treatment to delivery). We have only found this characteristic in Wöhlk and Laporte (2022) and Belenguer et al. (2024).

The SPDP was first introduced by De Meulemeester et al. (1997). They study the movement of full and empty skips in a setting with two types of customers and one site for skip disposal and they use vehicles of capacity one. They apply both Branch and Bound and heuristics to solve the problem. Several papers have since studied SPDP with capacity one. Belenguer et al. (2024) provide an overview and classification of SPDPs, which we repeat in Table 1.

A few papers stand out by having a capacity of two, as in our paper. Archetti et al. (2005) study a problem in which skips must be delivered from a central depot to a set of customers by use of vehicles of capacity

one and two. Their focus is on complexity results for this problem, which is significantly simpler than the ones we study.

Raucq et al. (2019) study a problem where full and empty skips are moved between customers, treatment facilities, and storage facilities by vehicles of capacity two. They apply a Column Generation approach to solve the problem. This problem is related to the drayage problem with combination vehicles. For a survey of that problem we refer the reader to the recent paper by Bustos-Coral and Costa (2025).

The SDPD with triplets of events and capacity two was first introduced by Wöhlk and Laporte (2022). They provide models for the SPDP-R and SPDP-F and present a Variable Neighborhood Search for them. Furthermore, they provide benchmark data from four waste collection companies in Denmark. Belenguer et al. (2024) focus on the SPDP-R. They develop several classes of valid inequalities, which they integrate in a Branch and Cut algorithm.

2.2. Fragment-based modeling

The method of fragments was first proposed by Alyasiry et al. (2019) and first applied to the PDP with time windows and last-in-first-out loading (PDPTWL). In that paper, a fragment is defined as a route where the vehicle is never empty, except at the beginning and end. There are only a few specialized PDP problems in which it is possible to enumerate all fragments. Various improvements have been proposed to overcome this problem and make the method applicable to wider classes of vehicle routing problems.

Rist and Forbes (2021) apply fragments to the Dial-a-Ride-Problem (DARP). They show that by changing the definition of a fragment to be a route where all pickups precede all deliveries, it is possible to enumerate all fragments for many more instances of the problem. They then define a network in which the arcs are derived from these “restricted fragments” and in which the nodes include state information indicating which requests are still on board. This network then forms the basis of a Branch and Cut algorithm which is the current state-of-the-art for the DARP.

Rist and Forbes (2022) introduce the concept of an “extended fragment”. In earlier fragment work the network consists of fragment arcs (from pickup nodes to delivery nodes) and linking arcs (from delivery nodes to pickup nodes). Rist and Forbes (2022) extend the fragments to pickup nodes. They show that while this increases the number of variables, it reduces the number of constraints, tightens the LP relaxation and allows for tighter valid inequalities, all of which leads to reduced run times.

More recently Sippel and Forbes (2024) have proposed several enhancements which aim to make the method of fragments applicable to wider classes of vehicle routing problems. These enhancements include “formulation leveraging” in which the tight LP relaxation of a route-based formulation is “leveraged” to filter fragments from the fragment formulation. The improved method outperforms state-of-the-art Branch and Price on a Pickup and Delivery problem and a truck and drone Vehicle Routing problem.

In a related method Dollevoet et al. (2020) propose a “p-step” method in which all fragments with p stops are enumerated and used to solve the Capacitated Vehicle Routing Problem.

3. Problem description

We are given a set of n requests $R = \{1, \dots, n\}$, where each request r is for q_r skips to be picked up (p_r) at location α_r , be treated (t_r) at location β_r , and delivered (d_r) at location γ_r . With each request is also associated a skip type ϕ_r . Let $F = \bigcup_{r \in R} \{\phi_r\}$ be the set of skip types. Let $L^P = \bigcup_{r \in R} \{\alpha_r\}$, $L^T = \bigcup_{r \in R} \{\beta_r\}$, $L^D = \bigcup_{r \in R} \{\gamma_r\}$, be the set of all pickup, treatment, and delivery locations, respectively. For the instances considered in this paper, $L^P = L^D$. Let 0 represent the location of the artificial depot, and define the full set of locations $L = L^P \cup L^T \cup L^D \cup \{0\}$. For any two locations $i, j \in L$, there is an associated travel cost c_{ij} and travel

Table 1

Skip Pickup and Delivery problems related to our work. MC: multi-commodity, TW: time windows, DR: driver rest time, PC: precedence constraints, HF: heterogeneous fleet, E: exact, H: heuristic. Source: Belenguer et al. (2024).

Reference	Vehicle capacity	Disposal sites	Return	Constraints	Solution method
De Meulemeester et al. (1997)	1	1	No	MC	E, H
Bodin et al. (2000)	1	1	No		H
Aringhieri et al. (2004)	1	Many	No		H
Archetti and Speranza (2005)	1	1	No		H
Archetti et al. (2005)	2	Many	No		H
Baldacci et al. (2006)	1	Many	No		E
Benjamin and Beasley (2010)	1	Many	No	TW, DR	H
Wy et al. (2013)	1	Many	No	TW, DR, MC	H
Rabbani et al. (2016)	1	1	No		H
Li et al. (2018)	1	Many	No	TW, PC	E, H
Raucq et al. (2019)	2	Many	No	TW, MC, HF	E, H
Wöhlk and Laporte (2022)	2	Many	Yes		H
Belenguer et al. (2024)	2	Many	Yes		E

time t_{ij} . Since the vehicles perform open routes, $c_{0j} = t_{0j} = c_{j0} = t_{j0} = 0$ for all locations $j \in L$. The triangle inequality is not respected in Wöhlk and Laporte (2022)) so we assume the same. We also assume symmetric travel times and costs.

We consider two versions of the problem as follows. In the fixed return version (SPDP-R), $\gamma_r = \alpha_r$ and $|F| = 1$. That is, every skip is returned to its origin and hence they can all be considered as of the same type. To state the flexible version (SPDP-F), we instead define S_r as the set of skips in request r and $r(s)$ as a mapping from S_r to R . We let γ'_s be a variable representing the delivery location of skip s , $\forall r \in R, s \in S_r$, and require that $\sum_{r \in R, \alpha_r = l, \phi_r = f} q_r = |\{s \in \bigcup_{r \in R} S_r : \gamma'_s = l \wedge \phi_{r(s)} = f\}|$, $\forall l \in L^P, f \in F$. I.e. the number of skips of a given type f returned to a given location l must equal the number of skips of that type originating from that location. This causes a redefinition of R for the SPDP-F such that R is further separated by skip type and each $r \in R$ is defined by α_r, β_r , and ϕ_r (rather than only α_r and β_r). Consequently, q_r becomes smaller, but the total number of skip movements is unchanged.

To satisfy the requests, we assume that we have a sufficiently large set of vehicles K , each with fixed cost C when used. Each skip of each request must be assigned to a vehicle that can complete all events for all of its assigned skips within a fixed time limit T . Each vehicle has a capacity of two skips, but it is allowed to complete operations for on-board skips in any order. Each vehicle completes an open route starting at its first pickup and ending at its final delivery. Following the original problem description, each of the three events (pickup, treatment, and delivery) has a fixed operation time. The total of the three operation times is $\hat{T}$.

Let E^P, E^T , and E^D denote the events corresponding to picking up, treating, and delivering requests, respectively, and let $E = E^P \cup E^T \cup E^D$. We define the quantity for an event as q_e , where $q_e = 1$ if $e \in E^P$, $q_e = -1$ if $e \in E^D$, and 0 if $e \in E^T$, and we define the location associated with event e as l_e .

A route can be represented as a sequence of events $P = [e_1, \dots, e_m]$. For a route to be legal it must satisfy the follow constraints.

- **Total duration:** $\sum_{i=1}^{m-1} t_{l_{e_i}, l_{e_{i+1}}} + m\hat{T}/3 \leq T$,
- **Capacity:** $\sum_{j=1}^i q_{e_j} \leq 2 \quad \forall \quad 1 \leq i \leq m$,
- **Pairing:** $\sum_{j=1}^m (e_j = p_r) = \sum_{j=1}^m (e_j = t_r) = \sum_{j=1}^m (e_j = d_r), \forall r \in R$.
- **Precedence:** $\sum_{j=1}^i (e_j = p_r) \geq \sum_{j=1}^i (e_j = t_r) \geq \sum_{j=1}^i (e_j = d_r), \forall i, 1 \leq i < m, \forall r \in R$.

That is, the total travel time plus service time must be less than the time limit, the vehicle must never have more than two skips on board, the number of pickups, treatments, and deliveries of a request must be the same, and for the individual skip, pickup must precede treatments, which again must precede delivery.

The cost of a route is given by $C + \sum_{i=1}^{m-1} c_{l_{e_i}, l_{e_{i+1}}}$. Our goal is to plan the set of legal routes that minimizes total costs of all routes, consisting of both fixed vehicle costs and routing costs and satisfies all requests.

Note that multiple requests can share the same pickup location. Furthermore, while Wöhlk and Laporte (2022) and Belenguer et al. (2024) define requests as movements of a single skip, our requests are defined to cover multiple skips. We emphasize that our problem is indeed identical to the two references. The reason for the change in definition is that we realize that the instances have multiple identical skip movements and our model can utilize this fact.

4. Restricted fragment network formulation

Before defining out network formally, we provide the reader with an intuitive explanation on a simplified version. We define a fragment as a partial route that starts with a pickup and ends immediately before another pickup, and which does not have any pickups after any deliveries. By definition, there can be at most two pickups in a fragment and therefore, there are few enough to enumerate them. Such fragments can be arranged in a network where, for each request, there are a number of nodes, each representing what else is on the vehicle upon pickup of a unit of that request. That way, the arcs in the network represent the fragments and a route is a path through the network such that the corresponding fragments represent the route once concatenated.

While this gives the intuition of our approach, we found that our model becomes even tighter if we aggregate nodes representing pickups of different requests at the same location. We now formally define our network $G(\mathcal{N}, \mathcal{A})$.

4.1. Nodes

We define a set of nodes $\mathcal{N}$, each consisting of a pickup location $i \in L^P$ and state information. Since the capacity is 2, when we pickup a skip for request $r \in R$ there are three cases. We can have nothing on board or we can have a skip of request $r' \in R$ on board that requires treating and delivering or we can have a skip of request r' on board that only requires delivering.

For each pickup location $l \in L^P$, we therefore define the set $\mathcal{N}_l$ of nodes as follows: a single node with state information $(l, -, -)$; a node with state information $(l, \beta_{r'}, \gamma_{r'})$ for each $r' \in R, \alpha_{r'} \neq l$; and node with state information $(l, -, \gamma_{r'})$ for each $r' \in R, \gamma_{r'} \neq l$.

Finally, we define node 0 as an artificial depot node. The complete set of nodes is defined as $\mathcal{N} = \{0\} \cup \bigcup_{l \in L^P} \mathcal{N}_l$. The size of $\mathcal{N}_l$ is bounded from above by $|R| + |L^P|$. Hence, the size of $\mathcal{N}$ is bounded by $|L^P|(|R| + |L^P|) + 1$.

Table 2

The set of arcs $\mathcal{A} \setminus \mathcal{A}_0^+$. The listed fragments are defined for any $r \in R$, $\alpha_r = l$ unless $r = r'$ and $Q_r = 1$ in which case we cannot pick up a skip of request r if the only such skip is already on board. If $r' = r$ some fragments in the table becomes equivalent.

from \ to	$(l'', \beta_{r'}, \gamma_{r'})$	(l'', β_r, γ_r)	$(l'', -, \gamma_r)$	$(l'', -, \gamma_r)$	$(l'', -, -)$ or 0
$(l, \beta_{r'}, \gamma_{r'})$	(p_r, t_r, d_r)	$(p_r, t_{r'}, d_{r'})$	$(p_r, \{t_r, d_r, t_{r'}, d_{r'}\})$	$(p_r, \{t_r, d_r, t_{r'}, d_{r'}\})$	$(p_r, \{t_r, d_r, t_{r'}, d_{r'}\})$
$(l, -, \gamma_r)$		(p_r, d_r)	(p_r, t_r, d_r)	$(p_r, \{t_r, d_r\})$	$(p_r, \{t_r, d_r, d_{r'}\})$
$(l, -, -)$	$(p_r, \{t_r, p_{r'}\}, d_r)$	$(p_r, p_{r'}, t_{r'}, d_{r'})$	$(p_r, \{t_r, p_{r'}, t_{r'}, d_r\})$	$(p_r, \{t_r, p_{r'}, t_{r'}, d_{r'}\})$	$(p_r, t_r, d_r) (p_r, \{t_r, d_r, p_{r'}, t_{r'}, d_{r'}\})$

4.2. Arcs

We define a set of arcs $\mathcal{A}$ as the union of the starting arcs, $\{(0, (l, -, -)), \forall l \in L^P\}$, and the arcs in Table 2, which we will now explain. We need the following definition.

Definition 1. Given $S \subset \mathcal{N}$, define $\mathcal{A}_S = \{(i, j) \in \mathcal{A} : i \in S, j \in S\}$, $\mathcal{A}_S^- = \{(i, j) \in \mathcal{A} : i \in S, j \notin S\}$, and $\mathcal{A}_S^+ = \{(i, j) \in \mathcal{A} : i \notin S, j \in S\}$. When $S = \{i\}$, we simplify the notation to $\mathcal{A}_i^-$ and $\mathcal{A}_i^+$.

Each arc $a \in \mathcal{A} \setminus \mathcal{A}_0^-$ in Table 2 corresponds to a fragment. We will therefore abuse notation slightly and also use the arc a to refer to the fragment. However, the term fragment will be reserved to discussions regarding routes and sequences, while we will use arc when referring to the graph or models.

Every fragment starts with the pickup of a request $r \in R$ and ends immediately before the pickup of request r'' . Another request, r' , may also be involved, either onboard at the beginning of the fragment or picked up after r . Note that when $q_r > 1$, it is possible to have $r'' = r$ and/or $r' = r$. Each row corresponds to a state at the pickup of request r at location $l = \alpha_r$. This is the start state of the fragment. The columns indicate the potential end state of the fragment, where the vehicle is ready to pick up a request r'' at location $l'' = \alpha_{r''}$. The parentheses show the events of the fragment. The events in $\{\}$ can be permuted to some extent. To illustrate, we consider the fragments from $(l, -, \gamma_r)$ to $(l'', -, -)$. They are stated as $(p_r, \{t_r, d_r, d_{r'}\})$, $\forall r \in R, \alpha_r = l$. The possible fragments are as follows: $(p_r, t_r, d_r, d_{r'})$, $(p_r, t_r, d_{r'}, d_r)$, (p_r, d_r, t_r, d_r) , $\forall r \in R, \alpha_r = l$, where the first two are equivalent if $r' = r$.

Define δ_{ar} as the number of times a skip of request $r \in R$ is picked up on the fragment associated with arc $a \in \mathcal{A}$. By definition, any two fragments between the same pair of nodes for which δ_{ar} is identical for all $r \in R$, contain exactly the same events. Let ω_a and τ_a be the cost and travel time associated with fragment a (arc $a \in \mathcal{A}$), respectively, and let τ_a include $\hat{T}$ for each pickup event of the fragment. The arcs in $\mathcal{A}_0^-$ have $\omega_0 = C$ and $\tau_0 = 0$.

Definition 2 (Dominated fragment). A fragment a_1 dominates a fragment a_2 , written as $a_1 > a_2$, if they start and end in the same state nodes, $\delta_{a_1 r} = \delta_{a_2 r}$, $\forall r \in R$, $\tau_{a_1} \leq \tau_{a_2}$, and $\omega_{a_1} \leq \omega_{a_2}$.

Definition 3 (Equivalent fragment). Two fragments a_1 and a_2 are equivalent, written as $a_1 \equiv a_2$, if $a_1 > a_2 \wedge a_2 > a_1$.

Because the vehicle can only carry two skips at a time, all fragments can easily be enumerated. Among these, we only keep non-dominated fragments and one of any equivalent fragments, such that the remaining fragments are all non-dominated and non-equivalent. These fragments define the arcs in $\mathcal{A} \setminus \mathcal{A}_0^-$.

4.3. Equivalence to existing fragment methods

Due to the simple polynomial number of nodes and arcs in our network, we can describe all nodes and arcs by enumerating the possibilities. However, the final network is equivalent to a network created by applying the techniques in the fragment literature. Recall that in Alyasiry et al. (2019) a fragment is a route in which the vehicle is never empty. For the SPDP, this leads to too many fragments. Instead, we can

start by generating restricted fragments (Rist & Forbes, 2021), which are fragments in which all pickups occur before all deliveries. These fragments can be converted into restricted fragment arcs by the process used to create arcs in Rist and Forbes (2021), slightly modified due to the presence of treatments. We can then arrive at the final set of arcs $\mathcal{A}$ by extending the restricted fragment arcs to pickup locations as in Rist and Forbes (2022).

5. Model for the SPDP-R

Using the network described in Section 4, we can formulate our Restricted Fragment Model (RFM), which solves a relaxation of the SPDP-R. Note that subtour elimination constraints are not included in the model, as they are added during the solution process, as detailed in Section 7.

Define binary variables x_{ak} to represent the number of times arc $a \in \mathcal{A}$ is traversed by vehicle $k \in K$. Finally, let the binary variables z_k be equal to one if vehicle $k \in K$ is used and zero otherwise. In the model, the vehicle index is essential for two reasons. Firstly, because each request consists of multiple skips and because multiple requests can share the same pickup location, multiple vehicles can visit the same node in the network. In fact, they can do so multiple times. Secondly, the vehicle index allows us to enforce the maximum route duration by a simple constraint.

$$\min \sum_{k \in K} \sum_{a \in \mathcal{A}} \omega_a x_{ak} \quad (1)$$

subject to:

$$\sum_{k \in K} \sum_{a \in \mathcal{A}} \delta_{ar} x_{ak} = q_r, \quad \forall r \in R \quad (2)$$

$$\sum_{a \in \mathcal{A}_i^-} x_{ak} = \sum_{a \in \mathcal{A}_i^+} x_{ak}, \quad \forall k \in K, i \in \mathcal{N}. \quad (3)$$

$$\sum_{a \in \mathcal{A}} \tau_a x_{ak} \leq T z_k, \quad \forall k \in K. \quad (4)$$

$$z_k = \sum_{a \in \mathcal{A}_0^-} x_{ak}, \quad \forall k \in K. \quad (5)$$

$$z_k \leq z_{k-1}, \quad \forall k \in K, k \neq 1. \quad (6)$$

$$x_{ak} \geq 0, \text{ integer} \quad \forall k \in K, a \in \mathcal{A} \quad (7)$$

$$z_k \in \{0, 1\} \quad \forall k \in K \quad (8)$$

The objective (1) is to minimize the sum of travel costs between locations. Constraints (2) ensure that every request is satisfied the correct number of times. Constraints (3) maintain the conservation of flow throughout the network. Constraints (4) ensure that the total time limit is not exceeded by any vehicle. Constraints (5) ensure that the number of vehicles leaving the depot is the same as the number of vehicles being used for the solution. Constraints (6) break symmetry and are not necessary for the feasibility of the model. Finally, Constraints (7)–(8) define the domain of the variables.

Table 3
Data for example.

Request	α_r	β_r	γ_r	L_{ϕ_r}
r	1	6	1	{1, 4}
r'	2	3	2	{2, 5}

6. Model for the SPDP-F

In this section, we explain the limited changes in RFM necessary to model flexible return.

For each skip type, $f \in F$, we define $R_f \subseteq R$ as the set of requests r with $\phi_r = f$ and $L_f \subseteq L^D$ as the set of potential delivery locations for skip type $f \in F$, i.e. $L_f = \bigcup_{r \in R, \phi_r = f} \gamma_r$. Finally, we define balance parameters $b_l^f = \sum_{r \in R, \phi_r = f, \alpha_r = l} q_r, \forall l \in L^D, f \in F$ as the number of skips of type f originating from location l .

For each fragment starting with the pickup of a skip of request r and potentially with a skip of request r' on the vehicle, as defined in Section 4.2, we duplicate it for each location in $L_{\phi(r)}$ and $L_{\phi(r')}$. We illustrate this by an example.

Assume that the data for r and r' are as stated in Table 3 and consider the fragment $(p_r, t_r, t_{r'}, d_{r'})$ from node $(\alpha_r, \beta_r, \gamma_r)$ to node $(l'', -, \gamma_r)$. For SPDP-R, this fragment follows the path $(1 \rightarrow 6 \rightarrow 3 \rightarrow 2 \rightarrow (l'', -, 1))$. For the SPDP-F, we create the four copies of the fragment, following the paths: $(1 \rightarrow 6 \rightarrow 3 \rightarrow 2 \rightarrow (l'', -, 1))$, $(1 \rightarrow 6 \rightarrow 3 \rightarrow 2 \rightarrow (l'', -, 4))$, $(1 \rightarrow 6 \rightarrow 3 \rightarrow 5 \rightarrow (l'', -, 1))$, $(1 \rightarrow 6 \rightarrow 3 \rightarrow 5 \rightarrow (l'', -, 4))$.

For each skip type $f \in F$ and each potential delivery location $l \in L_f$ for f , we define a parameter λ_{afl} for each arc $a \in \mathcal{A}$, to be the number of skips of type f that are delivered to location l by the fragment copy associated with a .

With requests copied as explained above and with the redefinition of R for SPDP-F explained in Section 3, the relaxed SPDP-F is solved by model (1)–(8) with the addition of Constraints (9), which ensure skip balance as defined in Section 3

$$\sum_{k \in K} \sum_{a \in \mathcal{A}} \lambda_{afl} x_{ak} = b_l^f \quad \forall f \in F, l \in L_f \quad (9)$$

7. Algorithmic details

To solve the SPDP-R and SPDP-F based on the RFM ((1)–(8) and (1)–(9), respectively), we have developed a Branch and Cut algorithm in which some additional constraints are only added in the root of the search tree, while others are added throughout the search. In this section, we provide the main elements of our Branch and Cut algorithm.

7.1. Branch and cut

As many skips can be picked up at one location, we expect cycles to appear in our solution. However, for a given solution to RFM disconnected cycles can appear in the arcs selected for a vehicle. Such a solution is not feasible and cannot be resolved by standard sub-tour elimination constraints due to the multi-visit nature of the problem. Instead, disconnected cycle elimination constraints are separated and added at the root node and at every integer solution encountered during the Branch and Bound process.

Specifically, we look for a set of nodes $S \subset \mathcal{N} \setminus \{0\}$ and a vehicle $k \in K$, for which condition (10) holds.

$$\sum_{a \in \mathcal{A}_S} x_{ak} > 0 \wedge \sum_{a \in \mathcal{A}_S^+} x_{ak} = 0 \quad (10)$$

Observing that $\tau_a > 0$ for any arc $a \in \mathcal{A} \setminus \mathcal{A}_0^-$, then for any S and k satisfying (10), the following constraint is violated:

$$\sum_{a \in \mathcal{A}_S \cup \mathcal{A}_S^- \cup \mathcal{A}_S^+} \tau_a x_{ak} \leq T \sum_{a \in \mathcal{A}_S^+} x_{ak}. \quad (11)$$

We therefore add (11) to the model for all $k \in K$. Adding this constraint ensures that the total travel time for vehicle k on arcs which touch some set of nodes S is limited to zero if vehicle k never travels from a node outside S to a node in S . Otherwise the overall travel time limit for vehicle k applies.

Disconnected subsets of nodes can be efficiently identified even for fractional solutions. These will always correspond to a violated constraint (11), so we check for these constraints at the root node of the IP solution process.

At the root node we also consider all sets S which consist of just the head and tail node of any arc a where $x_{ak} > 0$ for some k . This accounts for a pair of arcs connecting two nodes, where the two nodes are only minimally connected to other nodes. It also accounts for arcs that start and end at the same node, where the node is only minimally connected to other nodes.

For these smaller sets, which may not be disconnected, we add violated constraints (11). A time-independent condition can also be checked. As a result of the coverage requirements, each arc has a maximum allowable number of traversals. We define the maximum number of times an arc a can be used as M_a . Then for any set of nodes $S \subset \mathcal{N} \setminus \{0\}$ and any vehicle k the following constraint applies:

$$\sum_{a \in \mathcal{A}_S} x_{ak} \leq \left(\sum_{a \in \mathcal{A}_S} M_a \right) \left(\sum_{a \in \mathcal{A}_S^+} x_{ak} \right). \quad (12)$$

Violated constraints (12) are separated for these smaller sets S and added at the root node if the corresponding constraint (11) was not violated.

7.2. Minimize vehicles

In practice the fixed cost of vehicles is high, so the objective is always to first minimize the number of vehicles, then minimize the total travel cost, making the objectives lexicographic in practice. As in Alyasiry et al. (2019) we introduce an algorithmic enhancement whereby we first solve the linear relaxation of the model with a modified objective function that minimizes the total number of vehicles in the solution:

$$\min \sum_{k \in K} z_k \quad (13)$$

The value of $\hat{v} = \lceil \sum_{k \in K} z_k \rceil$ acts as a lower bound for the optimal number of vehicles used in the integer model. We add the constraint $\sum_{k \in K} z_k = \hat{v}$ and solve the model. If the model is infeasible we increment $\hat{v}$ and solve the model again.

7.3. Symmetry reduction

A further algorithmic speed-up is to assign requests which consist of one skip ($q_r = 1$) to the first vehicles in sequence. The first such request is assigned to the first vehicle, the second such request to one of the first two vehicles, and so on. This reduces the solution space by eliminating some symmetry. If we assume, without loss of generality, that the single requests are indexed first in R then this can be written as:

$$\sum_{k > r} \sum_{a \in \mathcal{A}, \delta_{ar} = 1} x_{ak} = 0, \quad \forall r \in R, r \leq |K|, q_r = 1. \quad (14)$$

7.4. Subset row inequalities

For requests where $q_r \geq 3$ and odd, we often see solutions to the linear relaxation where an arc variable is set to $q_r/2$, or where a collection of arc variables with $\delta_{ar} = 2$ sum to $q_r/2$. We can cut off such solutions with the following valid inequalities:

$$\sum_{k \in K} \sum_{a \in \mathcal{A}, \delta_{ar} = 2} x_{ak} \leq \lfloor \frac{q_r}{2} \rfloor, \quad \forall r \in R, q_r \geq 3, q_r \text{ odd}. \quad (15)$$

A similar situation can arise for multiple requests starting at the same location. Define $R_l = \{r \in R, \alpha_r = l\}$. Then we add the following valid

inequalities:

$$\sum_{k \in K} \sum_{a \in A_i} x_{ak} \leq \lfloor \frac{\sum_{r \in R_i} q_r}{2} \rfloor, \quad \forall i \in L^P, |R_i| > 1, \sum_{r \in R_i} q_r \geq 3 \text{ and odd.} \quad (16)$$

Finally we consider all subsets of two or three requests that contain a total of three skips and share the same pickup or treatment location. At most one arc can cover two of these requests. More formally, for sharing the same pickup location we have:

$$\sum_{k \in K} \sum_{a \in A_i} x_{ak} \leq 1, \quad \forall R' \subset R, |R'| > 1, \quad (17)$$

$$\sum_{r \in R'} q_r = 3, (\alpha_{r_1} = \alpha_{r_2} \forall r_1, r_2 \in R').$$

For sharing a treatment location, we replace the last condition with $\beta_{r_1} = \beta_{r_2} \forall r_1, r_2 \in R'$.

All inequalities in this section are added to the model before we solve the linear programming relaxation to minimize vehicles.

8. Results

We tested our algorithm on the SPDP benchmark data set first presented in Wøhlk and Laporte (2022). This data set is available at: <https://www.optimization.dk>. For our experiments we used the upper bounds from Wøhlk and Laporte (2022) as cutoff values, as was also done in Belenguer et al. (2024), which is the paper we compare to, thus ensuring fair comparison.

We limited our tests to instances where a solution with five or fewer vehicles has previously been found. These instances are listed in Table 4. For each instance we list the number of skips, the number of pickup locations $|L^P|$, the number of skip types $|F|$, the number of requests $|R|$, the number of nodes $|N|$, and the number of arcs $|A|$. These last three numbers are listed separately for SPDP-R and SPDP-F. We note that the number of requests is significantly lower than the number of skips for both SPDP-R and SPDP-F. Moreover, the number of arcs in SPDP-F is significantly more than the number of arcs in SPDP-R. This is more significant when $|F|$ is small (see for example B14).

Our code was written in Python 3.10.9 using Gurobi 12.0.0. Based on initial testing, the Gurobi parameter MIPFocus was set to 2 and BranchDir was set to 1. The instances listed in Table 5 were used for preliminary tuning of Gurobi parameters and for determining that the cuts listed in Section 4.2 were useful. All results are from overnight runs on a laptop with an Intel i9, 2.5 GHz, 14 core processor, with 64 GB RAM. For comparison, the results in Belenguer et al. (2024) were executed on an Intel Xeon, 3.5 GHz, 12 cores processor, with 64 GB RAM, using CPLEX 12.8.

Overall our run times and bounds are significantly better than Belenguer et al. (2024) and Wøhlk and Laporte (2022). We ran our code on a single thread for comparison with Belenguer et al. (2024) on the SPDP-R instances with up to 20 requests (17 instances). These results are provided in Table 5. For the eight instances where both methods found optimal solutions, our run times are faster by a factor of 35. For the remaining nine instances, our run times for solving the problems range between 2.8 seconds and 14 minutes, while the benchmark algorithm could not prove optimality within an hour (indicated by a “–” in the CPU column). Of note, by comparing the benchmark LB in Table 5 to our LP relaxation values (LB_r) in column 3 of Table 6, it can be observed that the gaps reported for the unsolved instances in the benchmark are often (much) bigger than the gap between our LP relaxation and the optimal solution.

We also ran all 49 SPDP-R and SPDP-F instances considered with 8 threads. The results are listed in Table 6. For the SPDP-R, we have solved 31 instances to optimality (23 of them for the first time) and additionally found improved solutions for 10 instances. For the

Table 4

Characteristics of the instances.

Inst	Skips	SPDP-R					SPDP-F		
		$ L^P $	$ F $	$ R $	$ N $	$ A $	$ R $	$ N $	$ A $
A1	5	3	4	5	20	579	5	31	1169
A2	7	3	3	6	22	766	6	31	3459
A3	7	2	2	7	12	600	7	33	3015
A4	10	4	2	5	32	993	6	65	11,929
A5	10	3	4	8	26	1521	9	58	9347
A6	12	4	3	7	38	1708	9	81	24,834
A7	12	4	3	11	50	4067	12	113	44,973
A8	15	5	4	8	58	3108	10	96	23,141
A9	15	4	4	9	44	3028	11	85	22,236
A10	20	3	3	8	26	1485	10	43	11,017
A11	20	3	6	15	40	5106	16	73	33,298
A12	25	4	4	14	59	6601	16	81	64,452
A13	25	4	4	14	59	6566	18	117	88,004
A14	30	6	4	19	132	20,881	20	217	366,712
A15	30	5	7	19	102	16,013	22	171	228,972
A16	36	5	5	21	110	19,687	24	181	300,313
A17	37	5	4	20	106	17,825	23	171	315,890
A18	38	5	5	16	90	11,990	19	171	178,474
A19	40	5	6	26	130	29,747	29	221	456,220
A20	49	5	5	21	110	20,044	28	176	432,173
B1	18	6	6	13	102	9373	16	145	100,971
B2	18	6	5	12	97	7461	15	133	96,836
B3	22	6	4	12	97	7647	15	133	126,362
B4	22	6	6	11	92	6931	17	145	105,969
B5	25	6	6	13	102	8853	18	169	110,696
B6	25	7	5	9	104	5629	17	134	180,654
B7	30	7	6	14	134	12,785	22	197	308,373
B8	30	8	5	17	184	22,609	24	321	708,847
B9	35	7	6	15	140	14,533	25	225	516,014
B10	35	7	6	17	152	19,042	24	281	535,333
B11	40	7	5	19	164	22,834	29	218	876,449
B12	40	8	6	19	198	28,058	30	337	945,180
B13	45	7	6	21	176	27,964	30	267	966,893
B14	45	8	4	25	240	46,774	32	385	2,352,351
B15	51	7	5	22	182	30,789	31	232	942,410
B16	51	7	7	25	200	38,851	36	267	1,604,103
B17	54	8	6	24	233	43,944	34	297	1,578,460
B18	57	7	6	21	176	28,396	36	323	1,707,264
B19	60	8	6	26	247	51,372	41	425	3,494,288
B20	63	7	6	23	188	33,230	39	253	1,761,933
C1	10	6	2	6	67	2116	8	97	31,950
C2	10	7	4	9	104	6174	9	183	52,799
C3	16	6	5	13	102	8893	15	139	62,830
C4	18	7	4	8	98	4437	10	106	34,101
C5	24	8	4	16	177	20,842	18	265	340,320
C6	24	7	5	16	146	17,258	17	267	245,783
C7	30	9	5	18	226	29,262	23	442	1,039,584
C8	34	9	5	18	226	30,192	24	433	812,681
D1	35	13	3	27	494	105,174	27	911	4,968,328

Table 5

Best single thread results compared to benchmark.

Inst	Benchmark		Results		
	LB	Gap	CPU	Opt	CPU
A1	661	–	0.1	661	0.1
A2	645	–	17.6	645	1.0
A3	628	–	8.6	628	0.1
A4	720	–	621.6	720	1.9
A5	724	–	48.9	724	2.9
A6	1240	–	73.1	1240	0.3
A7	737	40.9	–	1248	10.1
A8	1332.7	1.7	–	1356	17.6
A9	1253	–	930.4	1253	1.1
A10	1288	2.1	–	1315	6.1
A11	1294.5	3.5	–	1341	363.7
B1	1287.9	2.7	–	1323	797.8
B2	1125.3	2	–	1148	84.8
C1	894.3	34.3	–	1362	2.8
C2	1481	–	121.4	1481	45.0
C3	2271.4	2.6	–	2331	72.2
C4	1485.5	27.0	–	2034	140.8

Table 6

Bound and runtime information for both SPDP-R and SPDP-F.

No.	Fixed return							Flexible return						
	Cutoff	LB _r	LB _c	LB	UB	Gap	Time	Cutoff	LB _r	LB _c	LB	UB	Gap	Time
A1	661	646	660	661	661	*	0	648	634	648	648	648	*	0
A2	645	628	645	645	645	*	1	625	619	625	625	625	*	1
A3	628	621	625	628	628	*	0	619	619	619	619	619	*	0
A4	720	698	720	720	720	*	2	695	687	695	695	695	*	2
A5	724	703	724	724	724	*	5	707	691	702	707	707	*	10
A6	1240	1230	1240	1240	1240	*	1	1202	1200	1201	1201	1201	*	5
A7	1248	1226	1238	1248	1248	*	19	1211	1200	1210	1210	1210	*	11
A8	1356	1320	1343	1356	1356	*	24	1314	1297	1301	1314	1314	*	33
A9	1253	1237	1253	1253	1253	*	2	1244	1233	1243	1244	1244	*	28
A10	1316	1297	1305	1315	1315	*	8	1297	1284	1295	1297	1297	*	8
A11	1342	1312	1320	1341	1341	*	86	1289	1280	1282	1282	1282	*	12
A12	1935	1913	1919	1934	1934	*	100	1920	1892	1901	1901	1901	*	34
A13	1929	1905	1916	1927	1927	*	33	1879	1365	1368	1378	1378	*	711
A14	1979	1936	1954	1977	1977	*	1396	1951	1907	1914	1921	1921	*	1689
A15	2076	2059	2076	2076	2076	*	17	2076	2038	2045	2049	2053	0.2	TL
A16	2673	2632	2633	2643	2661	0.7	TL	2642	2085	2085	2089	2089	*	3099
A17	2679	2618	2621	2637	2642	0.2	TL	2638	2087	2087	2094	-	-	TL
A18	2695	2659	2667	2680	2690	0.4	TL	2682	2637	2645	2657	2657	*	1657
A19	2750	2685	2698	2707	2743	1.3	TL	2702	2629	2629	2631	2631	*	714
A20	2745	2705	2716	2722	2741	0.7	TL	2745	2684	2686	2689	2689	*	1581
B1	1323	1264	1308	1323	1323	*	264	1299	1247	1252	1291	1291	*	3360
B2	1148	1126	1138	1148	1148	*	75	1121	1112	1117	1118	1118	*	74
B3	1188	1159	1179	1185	1185	*	23	1163	1143	1145	1147	1147	*	448
B4	1178	1149	1168	1173	1173	*	33	1156	1134	1135	1141	1141	*	457
B5	1171	1141	1154	1166	1166	*	173	1166	1137	1139	1143	1143	*	910
B6	1205	1168	1193	1201	1201	*	20	1192	1162	1165	1166	1171	0.4	TL
B7	1761	1717	1718	1746	1746	*	1652	1751	1197	1201	1201	1201	*	652
B8	1759	1707	1725	1741	1741	*	2441	1739	1185	1187	1189	1193	0.3	TL
B9	1741	1698	1708	1727	1727	*	1972	1737	1690	1690	1694	1694	*	2125
B10	1846	1796	1796	1821	1821	*	449	1833	1773	1778	1778	1778	*	472
B11	1812	1764	1780	1791	1795	0.2	TL	1801	1737	1742	1742	1742	*	1971
B12	1934	1860	1885	1897	1897	*	407	1916	1845	1846	1846	1886	2.1	TL
B13	2393	1816	1830	1838	-	-	TL	2378	1790	1795	1795	1795	*	2442
B14	2390	2328	2338	2344	-	-	TL	2373	1794	1795	1796	-	-	TL
B15	2407	2353	2356	2368	2382	0.6	TL	2407	2318	2319	2319	2327	0.3	TL
B16	2416	2381	2394	2398	2410	0.5	TL	2415	2349	2350	2350	-	-	TL
B17	2391	2350	2367	2371	-	-	TL	2390	2316	2316	2317	-	-	TL
B18	2518	2459	2459	2474	2516	1.7	TL	2497	2426	2426	2426	-	-	TL
B19	3161	3076	3083	3090	-	-	TL	3158	3034	3034	3034	3053	0.6	TL
B20	3104	3031	3031	3037	3078	1.3	TL	3092	2992	2992	2993	-	-	TL
C1	1362	1318	1362	1362	1362	*	3	1297	1284	1296	1297	1297	*	8
C2	1481	1399	1481	1481	1481	*	39	1438	1396	1422	1438	1438	*	127
C3	2331	2198	2302	2331	2331	*	43	2269	2176	2238	2269	2269	*	141
C4	2034	1968	2010	2034	2034	*	29	1520	1456	1466	1495	1495	*	877
C5	2912	2820	2859	2912	2912	*	2777	2810	2270	2281	2288	-	-	TL
C6	3123	3025	3050	3076	-	-	TL	3055	2946	2954	2959	-	-	TL
C7	3137	2989	3036	3058	-	-	TL	2899	2803	2807	2807	-	-	TL
C8	3823	3690	3729	3750	-	-	TL	3568	2990	2991	2991	-	-	TL
D1	3402	2814	2842	2847	-	-	TL	2776	2674	2674	2674	-	-	TL

SPDP-F we have solved 32 of the 49 instances to optimality (30 of them for the first time) and found improved solutions for 6 additional instances.

Table 6 gives the following information for SPDP-R and SPDP-F for each instance: the best feasible solution from the literature, which we have used as a cutoff value (Cutoff); the value at the root node before (LB_r) and after (LB_c) adding cutting planes to the root; the best lower (LB) and upper (UB) after one hour with a '·' if no feasible solution is identified; the optimality gap computed as $\frac{UB-LB}{UB} * 100$ (Gap), which we substitute as * if optimality is proven; and the runtime in seconds, which includes both time for generating fragments, setting up the model, and running Gurobi. We use a runtime limit of one hour. We mark LB and UB in bold if it is better than the benchmark.

We make several observations relating to Table 6. The first is that the LP relaxation of RFM is generally tight. We observe longer run times on

either bigger instances, or instances with larger gaps between LB_c and LB. This is particularly notable for B1 which is a similar size to B2-B4, but has much higher run time for SPDP-R and SPDP-F. Additionally, the gap between LB_r and LB is generally smaller for SPDP-F. This appears to be the main reason instances A16 and A17-20 have solved to optimality for SPDP-F and not SPDP-R. We also observe a smaller gap between LB_r and LB_c for SPDP-F than for SPDP-R - an average gap of 6 compared to 20. Overall, this suggests that even though an instance of SPDP-F has more requests, nodes, and arcs than the corresponding SPDP-R instance, SPDP-F has a tighter LP relaxation.

Recall that the Cutoff columns are provided from the best solution found by the heuristic algorithm in Wöhlk and Laporte (2022). We note that these solutions are of very high quality for SPDP-R and that our algorithm only finds small improvements, and then mostly for larger cases. However it is a different story for SPDP-F. Here the gap between the best heuristic and the best solution from RFM (when it finds one)

Table 7
Number of cuts of each type added by our Branch and Cut.

No.	Fixed return					IPs	Flexible return					IPs
	(15)–(16)	(17)	(11)	(11)r	(12)r		(15)–(16)	(17)	(11)	(11)r	(12)r	
A1	1	1	7	2	4		1	1	3	2	0	
A2	0	4	17	32	23		0	4	3	0	0	
A3	1	5	5	3	6		1	5	13	0	0	
A4	2	0	12	66	16		2	4	0	2	7	
A5	2	5	9	62	33		2	13	83	7	5	
A6	2	3	7	5	21		2	9	5	7	3	
A7	1	9	13	142	131		1	13	3	3	6	x
A8	1	2	28	155	129		2	8	101	43	22	
A9	1	4	6	16	8		1	14	23	49	40	
A10	4	3	6	102	42		6	3	0	24	41	
A11	0	24	122	85	61		0	36	12	8	10	
A12	5	15	14	150	110		5	26	13	8	35	
A13	3	14	6	177	121		2	45	62	26	10	
A14	5	13	61	268	186		4	25	22	38	48	
A15	5	21	5	193	175		4	59	477	67	137	
A16	4	30	289	93	172	x	5	57	26	25	21	
A17	4	22	66	153	226	x	5	41	0	34	38	X
A18	6	12	106	217	238		6	20	13	80	146	x
A19	5	57	144	153	160		3	93	31	18	62	
A20	8	14	27	191	240		9	80	17	31	94	
B1	4	5	50	181	104		2	46	1407	45	22	
B2	3	8	35	249	133		2	45	9	32	39	
B3	4	9	25	135	53		3	24	164	74	80	
B4	5	0	9	270	194		6	70	106	19	15	
B5	3	2	35	213	140		2	88	683	44	65	
B6	3	3	24	172	64		4	98	1712	63	43	
B7	7	4	188	104	90	x	4	175	31	37	87	
B8	2	21	247	248	116	x	3	133	185	26	52	
B9	3	13	97	139	81		3	147	38	21	23	
B10	5	13	139	33	42		4	80	21	18	16	
B11	7	16	139	162	172		3	166	39	24	173	
B12	8	10	95	213	165		7	450	19	47	63	
B13	7	22	28	152	144	X	6	167	23	49	121	
B14	8	33	71	491	524	x	9	221	9	14	15	X
B15	9	22	154	71	92		8	106	55	29	86	
B16	6	60	75	295	413		6	246	0	36	51	
B17	5	41	49	390	320		7	228	9	11	18	
B18	9	13	242	46	62		9	335	21	34	51	
B19	10	27	95	230	321		8	370	25	0	0	
B20	6	38	227	41	87		7	374	24	0	57	
C1	1	4	32	102	42		1	20	6	7	1	
C2	0	2	22	264	132		0	2	47	225	33	
C3	3	8	22	133	84		2	31	3	113	44	
C4	3	1	34	195	101	x	3	9	195	114	100	
C5	3	17	45	587	308	x	3	51	7	54	46	X
C6	3	16	65	305	316		3	22	40	145	128	
C7	5	23	32	409	332		5	146	0	22	20	
C8	6	9	74	314	227		3	78	0	38	9	X
D1	5	90	41	394	222	X	5	90	0	0	0	X

is generally larger, underlining the superiority of our method both in terms of upper and lower bounds. Moreover, in 5 instances this gap is more than 500, indicating that RFM has been able to find a solution with one fewer vehicle. Only for eight (resp. eleven) instances for the SPDP-F (resp. SPDP-R) in the larger range were we not able to either identify a better feasible solutions than the benchmarks or prove that the benchmark solution was optimal.

Table 7 provides information about the number of cuts added. We use r to indicate cuts that are added in the root node. The final column for each of the two problems has an 'x' in the few situations where the number of vehicles had to be increased. We point out that this only happened for 7 of the 49 instances for SPDP-R and 2 times for the SPDP-F. We indicate with an 'X' when the time limit was reached before the root node was solved and when the gap was large enough to indicate that an additional vehicle might be needed. This happened 7 times in total. We point the reader to the fact that 83% of the dynamically added cuts are added in the root node for SPDP-R, which supports the discussion above regarding the tightness of LB_c . For the SPDP-F, only 41% of the dynamically added cuts are added in the root node.

9. Conclusion

This paper presented a novel formulation for the SPDP based on the method of fragments. Computational results show that our formulation significantly improves upon current state-of-the-art algorithms. For the first time, we extend the method of fragments to vehicle routing problems with three events per request and with multiple copies of each request.

There are still many benchmark instances of the SPDP-R and SPDP-F which remain unsolved to optimality. In future research we intend to apply the formulation leveraging techniques of Sippel and Forbes (2024) to these problems and/or apply Branch and Price techniques. We note that the pricing problem is substantially easier to formulate on the restricted fragment network described in Section 4. Nevertheless, the pricing problem could still be very difficult as the state space must include how many times each request has been picked up, leading to an exponential number of undominated labels for each node in the network.

Our model is generic and can solve the SPDP-R or SPDP-F by the addition/removal of a single set of constraints. This is interesting because

the problems are structurally different in the sense that the delivery locations are essentially variables in the SPDP-F while they are parameters in the SPDP-R. Furthermore, consider a version of the problem in which the delivery locations are fixed, but differ from the pickup location. Our model would be able to solve this problem without any modification, as it never takes advantage of the fact that $\alpha_r = \gamma_r$. Additionally, as the recycling industry grows, new treatment facilities appear. This gives room for variations of the problem in which there are multiple choices for treatment facilities¹. Our model can also handle this variation with fixed and flexible return, by generating the fragments accordingly. Of course, this will increase run times as the number of fragments will grow. Finally, the model would be able to handle all the above versions of the problem with larger vehicle capacity without any model modifications, provided that the fragments are generated accordingly. However, again this is expected to increase the runtime due to the increased number of fragments resulting from the increased capacity. To conclude, besides being a strong modeling tool in terms of solution efficiency, the fragment-based model is highly flexible.

CRedit authorship contribution statement

Michael Forbes: Writing – review & editing, Writing – original draft, Supervision, Software, Methodology, Formal analysis, Conceptualization; **Thomas Whitaker:** Writing – original draft, Software, Methodology, Conceptualization; **Sanne Wöhlk:** Writing – review & editing, Writing – original draft, Methodology, Conceptualization.

Declaration of competing interest

There are no conflict of Interest.

Acknowledgement

The third author is partially funded by the Danish Council for Independent Research - Social Sciences. Project ‘ComRoute’ [grant number 3099-00003B]. This support is gratefully acknowledged.

Appendix A. Notation

Table A.8

Notation used for problem definition.

Symbol	Description
Parameters	
n	Number of requests
q_r	Number of skips in request $r \in R$
p_r, t_r, d_r	Pickup, treatment, and delivery action for request $r \in R$
$\alpha_r, \beta_r, \gamma_r$	Pickup, treatment, and delivery location for request $r \in R$
ϕ_r	Skip type of request $r \in R$
c_{ij}	Cost of traveling between locations i and j
t_{ij}	Travel time between locations i and j
C	Fixed cost for using each vehicle
$\hat{T}$	Total time for the pickup, treatment, and delivery operations for one skip
T	Total time available
Sets and mappings	
R	Set of requests, indexed by r
F	Set of skip types
L	Set of all locations
L^P, L^T, L^D	Set of Pickup, treatment, and delivery locations
K	Set of vehicles, indexed by k
S_r	Set of skips in request r , indexed by s
$r(s)$	mapping from S_r to R
γ_s	Variable representing delivery location of skip s for SPDP-F

Table A.9

Additional notation.

Symbol	Description
E	Set of all events
E^P, E^T, E^D	Set of Pickup, treatment, and delivery events
q_e	Parameter for adjusting load change of vehicle for event $e \in E$
P	Sequence of events representing a route

Table A.10

Additional notation used for modeling and solving the problem.

Symbol	Description
Sets	
$G(\mathcal{N}, \mathcal{A})$	Fragment graph on which the model is built
$\mathcal{N}_l$	Set of nodes related to pickup location $l \in L_P$
$\mathcal{A}$	Set of Set of fragments and set of arcs in the fragment graph
R_f	Set of requests r with $\phi_r = f, f \in F$
L_f	Set of potential delivery locations for skip type $f \in F$
Parameters	
δ_{ar}	Number of times a skip of request r is picked up on a fragment of $a \in \mathcal{A}$
b_l^f	Number of skips of type $f \in F$ originating from location $l \in L$
ω_a	Cost of fragment of $a \in \mathcal{A}$
τ_a	Time of fragment of $a \in \mathcal{A}$
$\lambda_{a/l}$	Number of skips of type $f \in F$ returned to location $l \in L^D$ by fragment $a \in \mathcal{A}$
$\hat{v}$	Lower bound on optimal number of vehicles
M_a	Big- M value associated with $a \in \mathcal{A}$
Variables of models	
x_{ak}	Number of times arc $a \in \mathcal{A}$ is traversed by vehicle $k \in K$
z_k	Binary variable representing if vehicle $k \in K$ is used
Generic set notation	
$\mathcal{A}_S$	$\{(i, j) \in \mathcal{A} : i \in S, j \in S\}, S \subset \mathcal{N}$
$\mathcal{A}_S^-$	$\{(i, j) \in \mathcal{A} : i \in S, j \notin S\}, S \subset \mathcal{N}$
$\mathcal{A}_S^+$	$\{(i, j) \in \mathcal{A} : i \notin S, j \in S\}, S \subset \mathcal{N}$

In this appendix, we provide an overview of the notation used in the paper. For ease of reading, the notation is partitioned into three tables. Table A.8 provides the main notation used for the description of the problem. Table A.9 provides additional notation used in the description of the problems. Finally, Table A.10 provides the notation used in the models of the problems.

References

- Alyasiry, A., Forbes, M., & Bulmer, M. (2019). An exact algorithm for the pickup and delivery problem with time windows and last-in-first-out loading. *Transportation Science*, 53 (6):1695–1705. <https://doi.org/10.1287/trsc.2019.0905>.
- Archetti, C., Mansini, R., & Speranza, M. G. (2005). Complexity and reducibility of the skip delivery problem. *Transportation Science*, 39 (pp. 182–187). <https://doi.org/10.1287/trsc.1030.0084>.
- Archetti, C., & Speranza, M. G. (2005). Collection of waste with single load trucks: A real case. In B. F. et al. (Ed.), *Distribution logistics* (pp. 105–119). Springer-Verlag Berlin Heidelberg.
- Aringhieri, R., Bruglieri, M., Malucelli, F., & Nonato, M. (2004). An asymmetric vehicle routing problem arising in the collection and disposal of special waste. *Electronic Notes in Discrete Mathematics*, 17 (pp. 41–47).
- Baldacci, R., Bodin, L. D., & Mingozzi, A. (2006). The multiple disposal facilities and multiple inventory locations rollon-rolloff vehicle routing problem. *Computers & Operations Research*, 33 (pp. 2667–2702). <https://doi.org/10.1016/j.cor.2005.02.023>.
- Battarra, M., Cordeau, J. F., & Iori, M. (2014). Pickup-and-delivery problems for goods transport. In *Vehicle routing: Problems, methods, and applications*. 2nd edition. (pp. 166–191). Paolo Toth and Daniele Vigo.
- Belenguer, J., Cubillos, M., & Wöhlk, S. (2024). A branch-and-cut algorithm for a skip pick-up and delivery problem. *Computers & Operations Research*. 168 106705. <https://doi.org/10.1016/j.cor.2024.106705>.
- Benjamin, A. M., & Beasley, J. E. (2010). Metaheuristics for the waste collection vehicle routing problem with time windows, driver rest period and multiple disposal facilities. *Computers & Operations Research*, 37, (pp. 2270–2280). <https://doi.org/10.1016/j.cor.2010.03.019>.
- Berbeglia, G., Cordeau, J. F., Gribkovskaia, I., & Laporte, G. (2007). Static pickup and delivery problems: A classification scheme and survey. *TOP*. 15 (1), 1–31 <https://doi.org/10.1007/s11750-007-0009-0>.

¹ Such instances can be provided upon request.

- Bodin, L. D., Mingozzi, A., Baldacci, R., & Ball, M. O. (2000). The rollon-rolloff vehicle routing problem. *Transportation Science*, 34, (pp. 271–288). <https://doi.org/10.1287/trsc.34.3.271.12301>.
- Bustos-Coral, D., & Costa, A. M. (2025). Adaptive large neighborhood search for drayage routing problems involving longer combination vehicles. *Computers & Operations Research*, 173, (p. 106826). <https://doi.org/10.1016/j.cor.2024.106826>.
- De Meulemeester, G., Louveaux, F. V., & Semet, F. J. (1997). Optimal sequencing of skip collections and deliveries. *The Journal of the Operational Research Society*, 48 (1), 57–64 <https://doi.org/10.1057/palgrave.jors.2600325>.
- Dollevoet, T., Munari, P., & Spliet, R. (2020). A p-step formulation for the capacitated vehicle routing problem. <http://hdl.handle.net/1765/123411>.
- Hess, C., Dragomir, A., Doerner, K., & Vigo, D. (2023). Waste collection routing: A survey on problems and methods. *Central European Journal of Operations Research* . 32, 399–434. <https://doi.org/10.1007/s10100-023-00892-y>.
- Li, H., Jian, X., Chang, X., & Lu, Y. (2018). The generalized rollon-rolloff vehicle routing problem and savings-based algorithm. *Transportation Research Part B: Methodological*, 113 (pp. 1–23). <https://doi.org/10.1016/j.trb.2018.05.005>.
- Parragh, S. N., Doerner, K. F., & Hartl, R. F. (2008). A survey on pickup and delivery problems, part II: Transportation between pickup and delivery locations. *Journal für Betriebswirtschaft*, 58 (pp. 81–117). <https://doi.org/10.1007/s11301-008-0036-4>.
- Rabbani, M., Tabrizi, F. H., & Farrokhi-Asi, H. (2016). A hybrid algorithm for solving a roll-on roll-off waste collection vehicle routing problem considering waste separation and recycling center. *International Journal of Applied Operational Research*, 6 (2) (pp. 19–31). <https://ijorlu.liau.ac.ir/article-1-514-en.pdf>.
- Raucq, J., Sörensen, K., & Cattrysse, D. (2019). Solving a real-life roll-on-roll-off waste collection problem with column generation. *Journal on Vehicle Routing Algorithms*, 2 (pp. 41–54). <https://doi.org/10.1007/s41604-019-00013-6>.
- Rist, Y., & Forbes, M. A. (2021). A new formulation for the dial-a-ride problem. *Transportation Science*. 55 (5): 1113–1135 <https://doi.org/10.1287/trsc.2021.1044>.
- Rist, Y., & Forbes, M. (2022). A column generation and combinatorial benders decomposition algorithm for the selective dial-a-ride-problem. *Computers & Operations Research*. 140 105649. <https://doi.org/10.1016/j.cor.2021.105649>.
- Sippel, L., & Forbes, M. (2024). Enhancements of fragment based algorithms for vehicle routing problems. *arxiv:2411.13151*, <http://arxiv.org/abs/2411.13151>.
- Wöhlk, S., & Laporte, G. (2022). Transport of skips between recycling centers and treatment facilities. *Computers & Operations Research*. 145 105879. <https://doi.org/10.1016/j.cor.2022.105879>.
- Wy, J., Kim, B. I., & Kim, S. (2013). The rollon-rolloff waste collection problem with time windows. *European Journal of Operational Research*, 224, (pp. 466–476). <https://doi.org/10.1016/j.ejor.2012.09.001>.